# egui_typst

## Typst math and markup in egui

A manual: how to use it, how to choose and tune it, and the mathematics of what it draws.

Version 0.0.1
egui 0.37 · Typst 0.15
github.com/nihilist-narwhal/egui-typst
MIT or Apache-2.0

**Four kinds of box run through this manual.**

| | |
|---|---|
| **Tip** | something to do, or a better way to do it. |
| **Pitfall** | something that goes wrong, and why. |
| **The math** | the reasoning behind a behaviour, with the formulas. |
| **Measured** | a number from a test or an example in this repository. |

# Contents

1 What this is ..... 3
2 Quick start ..... 3
2.1 Add the crate ..... 3
2.2 The smallest application ..... 3
2.3 See it before you use it ..... 4
3 Writing snippets ..... 4
3.1 The page a snippet lives on ..... 4
3.2 A cheat sheet for actuarial formulas ..... 4
3.3 Two traps worth learning early ..... 5
3.4 Text you do not control ..... 5
3.5 What fonts are available ..... 5
4 Choosing a backend ..... 6
4.1 How they look at the pixel level ..... 6
4.2 Which one to use ..... 6
4.3 What things cost on one machine ..... 7
5 Theme, scale and caching ..... 7
5.1 Colour ..... 7
5.2 Scale and zoom ..... 7
5.3 What is cached, and what happens when it fills ..... 7
5.4 Threads ..... 8
6 Errors, limits and hostile input ..... 8
6.1 What can go wrong ..... 8
6.2 The ranges ..... 8
6.3 Snippets are code ..... 9
7 Tools that ship with the crate ..... 9
8 From page to pixels: the mathematics ..... 10
8.1 Units and sizes ..... 10
8.2 Colour and transparency ..... 10
8.3 From glyphs to polygons (the vector backend) ..... 11
8.4 Triangles ..... 11
8.5 The feathered edge ..... 12
8.6 How these claims were measured ..... 14
9 Reference ..... 15
9.1 The public interface ..... 15
9.2 What the vector backend does not draw ..... 16
9.3 Known limitations and what is next ..... 16
9.4 Rebuilding this manual ..... 16

# 1 What this is

`egui_typst` draws a Typst snippet as an egui widget. A snippet is Typst markup, usually a formula such as

$$\mathrm{TVaR}_{\alpha}(L) = \frac{1}{1-\alpha}\int_{\alpha}^{1} \mathrm{VaR}_{u}(L)\,\mathrm{d}u \tag{1}$$

which is typeset by the real Typst compiler, compiled once, cached, and drawn on every frame. The formula takes its colour from the `Ui`, so light and dark themes work, and it is redrawn sharply when the UI is scaled or zoomed.

**The mental model has three parts.** A **snippet** (text) is compiled to a **page** (Typst lays it out on an auto-sized page: glyph outlines and a few rules). The page is then **drawn** by one of three backends that differ only in how they turn the page into pixels (Figure 1). Everything that follows is about choosing among them, keeping them fast and safe, and understanding what they do.

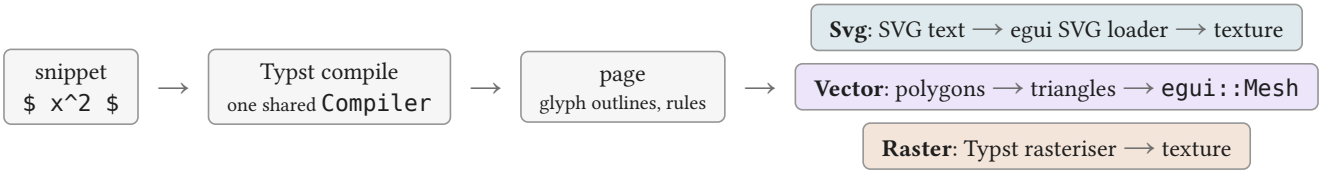



Figure 1: One compile, three ways to draw the result.

> **TIP**
>
> If you only read one chapter, read Section 2 and then Section 4. The default backend (`Svg`) is the right choice unless you have a reason to prefer another.

**Status.** The crate is experimental. It has three working backends, 29 tests that run without a GPU, and the safeguards of Section 6. It does not yet have text selection or accessible labels, and a snippet does not flow with surrounding egui text (Section 9.3).

# 2 Quick start

## 2.1 Add the crate

```toml
[dependencies]
egui = "0.37"
eframe = "0.37"
egui_typst = { git = "https://github.com/nihilist-narwhal/egui-typst" }
```

The crate is built on egui 0.37 and Typst 0.15. egui does not promise compatibility between minor versions, so an application on a different egui needs this crate on the same one.

## 2.2 The smallest application

```rust
use egui_typst::Typst;

struct App;

impl eframe::App for App {
    fn ui(&mut self, ui: &mut egui::Ui, _frame: &mut eframe::Frame) {
        ui.add(
            Typst::new(r#"$ "TVaR"_alpha (L) = 1/(1-alpha) integral_alpha^1 "VaR"_u
(L) dif u $"#)
                .size(18.0),
```

```
        );
    }
}

fn main() -> eframe::Result {
    eframe::run_native("demo", Default::default(), Box::new(|_| Ok(Box::new(App))))
}
```

`Typst::new` takes the snippet, `.size(pt)` the text size in points (default: the body size of the `Ui`), `.color(c)` the colour (default: the `Ui`'s text colour) and `.backend(b)` the backend (default `Svg`). Nothing else is needed: the compiler, its fonts and the caches are created on first use.

### 2.3 See it before you use it

```
cargo run --release --example gallery     # a window: seven formulas, backend switch,
zoom slider
cargo run --release --example render_png -- '$ x^2 $' x.png --dark --scale 3
```

> **TIP**
>
> The `gallery` example has a three-way backend switch and a zoom slider. Move the slider while switching backends: the differences Section 4 describes (what is recompiled, what stays sharp) become visible there.

## 3 Writing snippets

### 3.1 The page a snippet lives on

Every snippet is typeset on a page whose width and height are `auto` (it shrinks to the content), with a margin of 2 pt, in the colour and size you chose. Because the snippet is Typst **markup**, plain text, headings, emphasis and math all work; a formula is written between dollar signs.

```
$x^2$ is inline; the next is a display
formula: $ sum_(k=1)^n L_k $
```

$x^2$ is inline; the next is a display formula:

$$\sum_{k=1}^{n} L_k \tag{2}$$

Two spellings matter. A formula with spaces just inside the dollars, `$ ... $`, is a **display** formula (centred, larger operators); without them, `$...$` is **inline** (set in the line, as in a sentence).

### 3.2 A cheat sheet for actuarial formulas

| You type | You get | Note |
|---|---|---|
| `"TVaR"_alpha (L)` | $\mathrm{TVaR}_{\alpha}(L)$ | Words go in quotes: an unquoted word is looked up as a name and fails. |
| `EE[L_k \| L >= "VaR"_alpha]` | $\mathbb{E}[L_k \mid L \geq \mathrm{VaR}_{\alpha}]$ | `EE` is the expectation sign; `PP` is the probability sign. |
| `1/(1-alpha)` | $\frac{1}{1-\alpha}$ | Parentheses group the denominator and disappear. |
| `integral_alpha^1 f(u) dif u` | $\int_{\alpha}^{1} f(u)\,\mathrm{d}u$ | `dif` is the upright d of a differential. |
| `sum_(k=1)^n L_k` | $\sum_{k=1}^{n} L_k$ | Subscripts and superscripts of more than one token need parentheses. |
| `overline(x) quad hat(sigma)` | $\overline{x} \quad \hat{\sigma}$ | `quad` is a wide space. |

| | | |
|---|---|---|
| `mat(1, rho; rho, 1)` | $\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ | Commas separate columns, semicolons rows. |
| `sigma / mu approx "CoV"` | $\frac{\sigma}{\mu} \approx \text{CoV}$ | `approx`, `>=`, `<=`, `!=` are the usual relations. |
| `cases(0 "if" x < 0, 1 "if" x >= 0)` | $\begin{cases} 0 \text{ if } x < 0 \\ 1 \text{ if } x \geq 0 \end{cases}$ | For piecewise definitions. |

## 3.3 Two traps worth learning early

**Unquoted words.** In math mode a single letter is a variable and two side by side are a product, but a longer word is a *name* that Typst looks up (`alpha` is a Greek letter, `sum` an operator). `TVaR` is not a name, so the snippet does not compile: Typst reports `unknown variable: TVaR`, and the widget shows that message in place of the formula. Quote words:

```
$x y$ is a product; $"TVaR"_alpha$ is a word
```

$xy$ is a product; $\text{TVaR}_{\alpha}$ is a word

**Markup characters in text.** Outside math, `#` starts code, `$` starts math, `*` and `_` start emphasis, `@` starts a reference and `<...>` a label. Text containing them must be escaped.

## 3.4 Text you do not control

A snippet is not data, it is *code*: Typst can loop, recurse and compute. Text that comes from a user, a file or a network must therefore never be put into a snippet as it is. `egui_typst::escape` puts a backslash before every ASCII punctuation character, which Typst reads as "this character, literally":

```
use egui_typst::{escape, Typst};

let note = user_supplied_text();                    // may contain # $ * _ @ ...
ui.add(Typst::new(&escape(&note)));                 // shown literally, never run
ui.add(Typst::new(&format!("Loss ratio: ${}$", my_formula)));   // formulas come from you
```

**PITFALL**

`escape` is for markup text. Do not use it inside `$ ... $`, where the backslash has another meaning. And it protects against *code*, not against *length*: a very long escaped string is still a very long line.

**MEASURED**

The tests in `tests/escape.rs` escape every ASCII punctuation character, plus a hostile line made of `#read("/etc/passwd")`, math, emphasis, a reference, a label and a list marker. All of it renders. The same hostile line unescaped is an error (it tries to read a file). Escaped, `a\#b` is wider than `ab`: the `\#` is drawn, not run.

## 3.5 What fonts are available

Only the four families that ship with `typst-assets`: *Libertinus Serif*, *New Computer Modern*, *New Computer Modern Math* and *DejaVu Sans Mono* (17 faces). System fonts are not used.

**PITFALL**

Text in a script the fonts do not cover does not fail. It compiles, and draws the empty boxes of missing glyphs, because the compiler discards Typst's warnings. Latin, Greek, Cyrillic and Arabic draw; Chinese, Japanese, Korean and emoji draw boxes. If your users' text can contain them, check what you get before shipping.

# 4 Choosing a backend

The three backends draw the same page. They differ in what they keep, and therefore in what a change costs.

| | `Svg` (default) | `Vector` | `Raster` |
|---|---|---|---|
| What is kept | the page as SVG text | the page as polygons, then a mesh | pixels in a texture |
| Drawn by | egui's SVG loader (resvg) | egui, as a triangle mesh | egui, as a texture |
| New colour | swap a string: **0 compiles** | rebuild the mesh: **0 compiles** | recompile: **1 compile** |
| New scale or zoom | loader re-rasterises: **0 compiles** | rebuild the mesh: **0 compiles** | recompile: **1 compile** |
| Per-frame cost | draw one texture | copy and draw a mesh | draw one texture |
| Memory | a texture at the on-screen size | about 7,000 triangles for the TVaR formula | a texture at the on-screen size |
| Best for | almost everything | very large scale, no texture | a reference; comparisons |

Table 1: What each backend keeps and what a change costs. The compile counts are asserted by `tests/cache.rs`.

**MEASURED**

`tests/cache.rs` draws the same formula for four frames and then changes the colour, then the scale, and counts Typst compilations. `Svg`: 1, 0, 0. `Vector`: 1, 0, 0. `Raster`: 1, 1, 1. The count comes from `egui_typst::debug::compile_count`.

## 4.1 How they look at the pixel level

Figure 1 shows a small formula at 2 pixels per point through every backend, enlarged without smoothing so that individual pixels are visible. The three anti-aliased results are nearly identical; the one without the feathered edge (Section 8.5) shows why that edge exists.

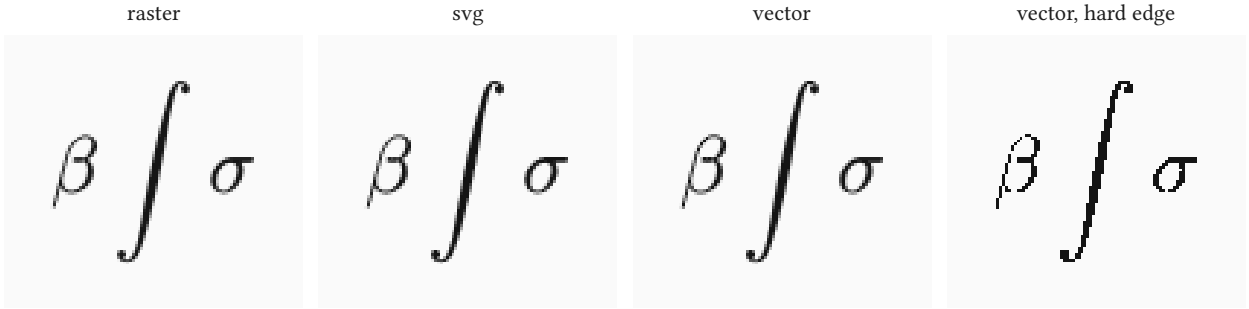



Figure 1: `$ beta integral sigma $` at 2 px/pt, each pixel enlarged. The right-hand image is the vector mesh without the feathered ring: edges are visibly stair-stepped.

## 4.2 Which one to use

- **Start with `Svg`.** Recolouring is free, zoom is sharp, and the work per frame is one textured quad. It is what the crate does when you say nothing.
- **Use `Vector`** when you want no texture at all (a custom renderer, very large zoom, formulas on a 3D surface) or when you want to inspect or transform the geometry. It costs more per frame, because the cached mesh is copied and translated each time it is drawn.
- **Use `Raster`** as a reference: it is Typst's own rasteriser, and the other two are tested against it. Its output is baked at one colour and one scale, so a theme switch or a zoom recompiles every formula on screen.

> **TIP**
>
> If your application has a theme switch or a zoom control, the table above is the argument for `Svg` or `Vector`: with `Raster` every formula on screen is recompiled the moment the user touches either.

### 4.3 What things cost on one machine

| Time | Step (release build, one machine; the spread from run to run is about 2×) |
|---:|---|
| ≈ 3 ms | `Compiler::new`: fonts and library, once per program |
| 0.5–1.2 ms | compile and rasterise a new snippet at 2× (`Raster`) |
| 0.6–0.9 ms | compile and export SVG for a new snippet (`Svg`) |
| ≈ 0.6 ms | compile and walk the frame for a new snippet (`Vector`) |
| 0.003–0.01 ms | recolour an SVG: the whole cost of a theme change under `Svg` |
| 0.9–2 ms | triangulate and feather the TVaR formula (about 6,700 triangles): the cost of a theme change under `Vector` |

> **PITFALL**
>
> These are indicative, not a benchmark. `cargo run --release --example timings` reproduces them. The "new snippet" is the same formula with a different trailing number, so Typst's memoisation still helps (the same snippet again takes 0.1–0.2 ms). A formula that shares nothing with what the process has seen costs more. Treat the compile counts above as the firm claim and the milliseconds as order-of-magnitude.

## 5 Theme, scale and caching

### 5.1 Colour

By default a snippet is drawn in `ui.visuals().text_color()`, so a theme switch needs no code. `.color(c)` overrides it. The colour's alpha is honoured by all three backends: `Raster` bakes it into the pixels, `Vector` multiplies it into the vertex colours, and `Svg` applies it as an image tint (the SVG itself carries the opaque colour).

> **TIP**
>
> Parts of a snippet that you colour yourself keep their colour. Only text in the snippet's own colour follows the theme. `#text(fill: red)[loss]` stays red in both themes; check it against both backgrounds.

### 5.2 Scale and zoom

All three backends draw at `ctx.pixels_per_point()`, which already includes egui's zoom factor (`ctx.set_zoom_factor`). The size you give with `.size(pt)` is in egui points, so a formula keeps its proportions to the surrounding text at any zoom.

### 5.3 What is cached, and what happens when it fills

| Backend | Cache keys (each cache holds at most 256 entries) |
|---|---|
| `Svg` | compiled page: snippet and size · tinted SVG bytes: snippet, size and RGB colour |
| `Vector` | polygons: snippet and size · meshes: snippet, size, RGBA colour and scale |
| `Raster` | textures: snippet, RGBA colour, size and scale |

When a cache reaches 256 entries and a new one arrives, **the whole cache is cleared** and refilled on demand.

> **PITFALL**
>
> A screen that shows more than 256 different formulas at once (or more than 256 combinations of colour and scale) will clear its cache on every frame, and so recompile every frame. The cap is a constant (`MAX_CACHED` in `src/widget.rs`); there is no setter. Paging or virtualising long lists of formulas keeps the visible count far below it.

### 5.4 Threads

`Compiler` is `Send + Sync` and may be shared. It has **one** main file, so one compile runs at a time: a gate is held for the whole compile.

> **MEASURED**
>
> This gate was missing in the first versions. `compile` swapped the snippet into the main file under a lock, released the lock, and *then* compiled, so a second thread could overwrite the text first. `tests/threads.rs` (eight threads, sixty renders each, snippets of clearly different widths) failed on 3 of 3 runs, for example a snippet that should be 168 px wide coming back 1191 px wide, which is another thread's picture. With the gate it passes on 5 of 5.

The widget itself compiles on the thread that calls `ui.add`, which in egui is the UI thread. For formulas that are many or heavy, compile them on a worker with a `Compiler` of your own (`Compiler::svg`, `Compiler::vector`) before they are needed.

## 6 Errors, limits and hostile input

### 6.1 What can go wrong

| `Error` | Meaning |
|---|---|
| `Compile(messages)` | the snippet is not valid Typst; one message per diagnostic |
| `Empty` | it typeset to no page |
| `Style(why)` | a size, margin or scale outside the ranges below (NaN and infinity included) |
| `TooLarge { width, height }` | the page would not fit the pixel budget below |
| `Setup(why)` | the compiler could not be built |

The widget never panics and never propagates an error: it draws the message in the error colour, in monospace, where the formula would have been.

### 6.2 The ranges

| Quantity | Allowed | Where |
|---|---|---|
| text size | 1 to 200 pt | `Style::size_pt`, the widget's `.size` |
| margin | 0 to 100 pt | `Style::margin_pt` (the widget uses 2 pt) |
| raster scale | 0.25 to 16 px/pt | `Style::pixels_per_point` |
| pixel budget | see Equation 3 | every backend, before anything is allocated |
| contours · points · segments per curve | 100,000 · 2,000,000 · 64 | `Vector` |

THE MATH

With scale $p$ (pixels per point) and a page of $W \times H$ points, the pixel budget is

$$\lceil pW \rceil \le 8192, \quad \lceil pH \rceil \le 8192, \quad \lceil pW \rceil \cdot \lceil pH \rceil \le 2^{25} \tag{3}$$

that is, at most 8192 pixels on a side and $2^{25}$ = 33,554,432 pixels in all (128 MiB of RGBA). It is checked with `ceil`, a little conservative: Typst itself rounds. `Svg` and `Vector` are bounded in points (8192) by the compiler, and the widget applies Equation 3 at the scale on screen before it hands a page to the SVG loader.

MEASURED

`#box(width: 1e6pt, height: 1e6pt)` once made `render` ask for a pixmap of about two million by two million pixels, 16 TB, and abort the whole process: Typst allocates the whole pixmap up front, and the size check came after. The check now comes first (`tests/limits.rs`, all three backends, plus a widget-level test that no texture is uploaded for a 5000 pt page).

### 6.3 Snippets are code

The compiler gives a snippet no files and no packages, so it cannot read your disk or the network. Typst also refuses runaway code. `cargo run --release --example hostile` runs ten hostile snippets, each on its own thread with a deadline:

| Snippet | Time | Result |
|---|---|---|
| `#while true { }` | 7 ms | error: *condition is always true* |
| `#let f(n) = f(n + 1) #f(0)` | 15 ms | error: *maximum function call depth exceeded* |
| `#read("/etc/passwd")` | 4 ms | error: file not found |
| `#import "@preview/cetz:0.3.0"` | 13 ms | error: file not found |
| `#box(width: 1e6pt, height: 1e6pt)` | 16 ms | error: too large (after the fix above) |
| `#range(100000000).len()` | **13 s** | renders |
| `#for i in range(10000) [ $x^#i$ ]` | **17 s** | error: too large, after compiling all of it |

PITFALL

**Typst has no time limit.** The two slow cases are not bugs in this crate: Typst is doing the work you asked for. But the widget compiles on the UI thread, so such a snippet freezes the interface for its duration, and it cannot be cancelled. Rules of thumb: formulas come from you, text from others goes through `escape`, and anything in between is compiled on a worker thread that you can abandon.

## 7 Tools that ship with the crate

| Command | What it is for |
|---|---|
| `--example gallery` | A window with a backend switch, a light/dark switch and a zoom slider. |
| `--example render_png` | Renders one snippet to a PNG without a window. `--backend raster\|svg\|vector`, `--scale N`, `--dark`; with `vector`, `--wire` overlays the triangles and `--hard` removes the feathered edge. Every figure in this manual that is a picture of output came from it (`manual/figures.sh`). |
| `--example timings` | The table in Section 4, on your machine. |

| | |
|---|---|
| `--example hostile` | The table in Section 6.3, on your machine. It takes about twenty seconds, by design. |
| `egui_typst::debug` | Hidden from the docs, useful in tests: `rasterise_alpha` draws an `egui::Mesh` on the CPU the way a GPU would without multisampling; `wire_edges` lists a mesh's triangle edges; `compile_count` counts compilations. |

**TIP**

No GPU? None of the 29 tests needs one. Running a frame with `ctx.run_ui(...)` returns the texture uploads and shapes egui *would* send to the renderer, so a test can look at the pixels an SVG loader produced (`tests/loader.rs`) or at the mesh a widget painted. That is how every claim in Section 8.6 was checked.

# 8 From page to pixels: the mathematics

This part explains what the backends do, with the formulas, and shows the measurements that confirm them. It is not needed to use the crate; it is what you need to change it, or to trust it.

## 8.1 Units and sizes

Typst measures in points (1 pt $= \frac{1}{72}$ in). egui measures in *points* too, and one egui point is `pixels_per_point` physical pixels. A page of $W \times H$ points therefore becomes a texture of

$$W_{\text{px}} = \lfloor pW \rceil, \quad H_{\text{px}} = \lfloor pH \rceil \tag{4}$$

pixels, where $p$ is the scale. The TVaR formula of Equation 1 at 14 pt typesets to a page of $223.7 \times 41.0$ pt; at $p = 2$ that is $447 \times 82$ px, which is exactly what `Raster` produces.

**THE MATH**

SVG has its own unit. A length in `pt` in an SVG file is read at 96 dpi by most renderers, so 1 pt $= \frac{96}{72}$ px $= \frac{4}{3}$ px. The widget never depends on this: it hands the SVG loader an exact size in egui points, and the loader scales the document to fit. The tests that rasterise an SVG directly have to undo it, rendering at $p \cdot \frac{3}{4}$ to compare at $p$ pixels per point.

## 8.2 Colour and transparency

egui textures and mesh vertices hold **premultiplied** colour: a pixel of colour $c$ and opacity $a$ is stored as $(ac, a)$. Drawing a source pixel $s$ over a destination pixel $d$ is

$$C_{\text{out}} = C_s + C_d(1 - \alpha_s), \quad \alpha_{\text{out}} = \alpha_s + \alpha_d(1 - \alpha_s) \tag{5}$$

where $C$ is the premultiplied colour and $\alpha$ the opacity. Typst's rasteriser (tiny-skia) already produces premultiplied pixels, so `Raster` hands them to egui unchanged.

**Recolouring an SVG without compiling again** relies on one fact: Typst writes a colour into the SVG as a hex literal, once per use. `Svg` compiles every snippet with the text colour `#010203`, a colour nothing else uses, and replaces that literal with the real colour when it draws:

```
svg.text.replace("#010203", "#rrggbb")
```

a string substitution over about 18 kB of text, which is the 3 to 10 microseconds measured above. A snippet that colours something `#010203` itself would be recoloured with the text; that is the one collision, and it is documented in `Svg::tinted`. The colour's alpha cannot live in the SVG that way, so it is applied as an image tint, a multiplication of every channel by the opacity.

## 8.3 From glyphs to polygons (the vector backend)

### *8.3.1 Where each glyph goes*

A run of text is a list of glyphs with advances. In font units a glyph outline has points $(X, Y)$ with $Y$ pointing up; the font has $u$ units per em, and the run has size $z$ points. Let $s = \frac{z}{u}$. For the $i$-th glyph, with pen position

$$x_i = \sum_{j<i} a_j + o_i^x, \quad y_i = \sum_{j<i} b_j + o_i^y \tag{6}$$

(advances $a_j, b_j$ and offsets $o_i$ in points), a point of the outline lands on the page at

$$(x, y) = (x_0 + x_i + sX, \quad y_0 - y_i - sY) \tag{7}$$

where $(x_0, y_0)$ is the origin of the run, itself reached through the transforms of the groups that contain it. The minus signs flip the font's upward $Y$ into the page's downward $y$.

### *8.3.2 Curves become polylines*

Outline curves are quadratic (TrueType) or cubic (CFF) Bézier curves; quadratics are raised to cubics, with $c_1 = p_0 + \frac{2}{3}(q - p_0)$ and $c_2 = p + \frac{2}{3}(q - p)$. A cubic with control points $P_0 \ldots P_3$ is cut into $n$ pieces at equal steps of its parameter. Let

$$M = \max(|P_0 - 2P_1 + P_2|, |P_1 - 2P_2 + P_3|) \tag{8}$$

be the larger second difference of the control points. The second derivative of the curve is

$$B''(t) = 6\big[(1-t)\Delta^2 P_0 + t\Delta^2 P_1\big] \tag{9}$$

so $|B''| \le 6M$. The chord of a piece of parameter length $\frac{1}{n}$ lies within $\frac{h^2}{8} \cdot \max|B''|$ of the curve (the linear-interpolation error bound with $h = \frac{1}{n}$), hence

$$\varepsilon \le \frac{3}{4} \cdot \frac{M}{n^2}, \quad \text{so} \quad n = \left\lceil \sqrt{\frac{3M}{4\varepsilon}} \right\rceil \tag{10}$$

For tolerance $\varepsilon = 0.01$ pt, a nearly straight piece needs one segment and a tight curve a few; `n` is capped at 64.

> **MEASURED**
>
> `vector::tests::flattening_stays_within_the_tolerance` flattens 3,000 random cubics in a 60 × 60 pt box (and checks 400 points of each against the polyline). Of the 949 that stay under the 64-segment cap, the worst error is **0.982 of the tolerance**; the other 2,051 are wilder than any glyph and hit the cap, where the tolerance is deliberately not promised (the cap bounds the work a hostile snippet can ask for). The first version estimated $M$ from the control points' distance to the chord instead; it undershot, by up to 1.22× on one curve. The rigorous bound costs the same.

### *8.3.3 Which side is the inside*

A glyph is a set of closed contours, filled with the *non-zero winding* rule, in which the outer contour and the holes wind in opposite directions. The signed area of a polygon with vertices $(x_i, y_i)$ is

$$A = \frac{1}{2} \sum_i (x_i y_{i+1} - x_{i+1} y_i) \tag{11}$$

and the sign of $A$ says which side of each edge is the interior: for $A > 0$ it is the side the vector $(-d_y, d_x)$ points to, for an edge with direction $(d_x, d_y)$. The code takes the sign of the contour of largest $|A|$ (the outline of the glyph) and uses it for every contour; holes then come out right because they wind the other way.

## 8.4 Triangles

egui can fill only convex shapes, and a letter is not convex: an "a" has a counter, an "S" has bays. The interior is therefore cut into triangles with the `lyon` tessellator, using the non-zero rule. The result is a list of vertices and triples of indices, which is exactly what an `egui::Mesh` is.



Figure 2: The letter α at 14 px/pt: two contours become 423 triangles (the interior, then the feathering ring along the outline). Right: the triangle edges drawn over the glyph (`--wire`).

## 8.5 The feathered edge

Nothing in egui anti-aliases a raw mesh. A GPU draws a triangle's pixel when the pixel's *centre* is inside it: no partial coverage, hence stair-steps (Figure 1, right). The backend adds its own anti-aliasing, as a thin ring around every contour.

### *8.5.1 The idea*

Let $s$ be the signed distance in points from a pixel centre to the true edge, positive outside the shape. For a straight edge parallel to a pixel row, the fraction of the pixel the shape covers is a linear ramp of width one pixel centred on the edge:

$$\alpha(s) = \operatorname{clamp}\left(\frac{1}{2} - ps, 0, 1\right) \tag{12}$$

with $p$ the scale (a pixel is $\frac{1}{p}$ points wide). The mesh reproduces this ramp: the opaque interior is **inset** by half a pixel, and a ring of quads runs from the inset line (opacity 1) to an **outset** line half a pixel outside (opacity 0). The GPU interpolates opacity across the ring, and a sample at distance $s$ from the true edge reads $\alpha(s)$ of Equation 12. The interior must be inset, not just surrounded by a ring: if the interior ran to the true edge, the ring's inner half would add nothing and the edge would come out about 0.125 px heavier on each side, which on a one-pixel stem is a visible 25%.

### *8.5.2 Where the offsets go*

At a vertex between edges with unit normals $n_0, n_1$ (pointing to the interior), the offset by distance $d$ follows the bisector

$$\hat{n} = \frac{n_0 + n_1}{|n_0 + n_1|} \tag{13}$$

and is lengthened so that both edges move by exactly $d$:

$$v' = v + \frac{d}{\cos\left(\frac{\varphi}{2}\right)}\hat{n}, \quad \cos\left(\frac{\varphi}{2}\right) = \hat{n} \cdot n_0 \tag{14}$$

where $\varphi$ is the angle between the normals. At a sharp corner $\cos\left(\frac{\varphi}{2}\right) \to 0$ and the offset would shoot off to infinity, so $\cos\left(\frac{\varphi}{2}\right)$ is clamped from below at $\frac{1}{2.5}$: no vertex moves more than $2.5d$.

### *8.5.3 How good is a linear ramp?*

Equation 12 is exact for an edge aligned with the pixel grid and approximate otherwise: the true coverage of a unit pixel by a tilted half-plane is not linear in $s$. For an edge at 45° and $t = -s$, the exact coverage is

$$c(t) = \begin{cases} 0 & \text{if } t \le -\frac{\sqrt{2}}{2} \\ \left(t + \frac{\sqrt{2}}{2}\right)^2 & \text{if } -\frac{\sqrt{2}}{2} < t \le 0 \\ 1 - \left(\frac{\sqrt{2}}{2} - t\right)^2 & \text{if } 0 < t \le \frac{\sqrt{2}}{2} \\ 1 & \text{if } t > \frac{\sqrt{2}}{2} \end{cases} \tag{15}$$

and the two curves are drawn in Figure 3. They agree at $s = 0$ (coverage 1/2) and differ most at $|s| = \frac{1}{2}$, where the ramp has already reached 0 or 1 and the exact curve has not: there the error is $\left(\frac{\sqrt{2}}{2} - \frac{1}{2}\right)^2 = \frac{(\sqrt{2}-1)^2}{4} \approx 0.043$.

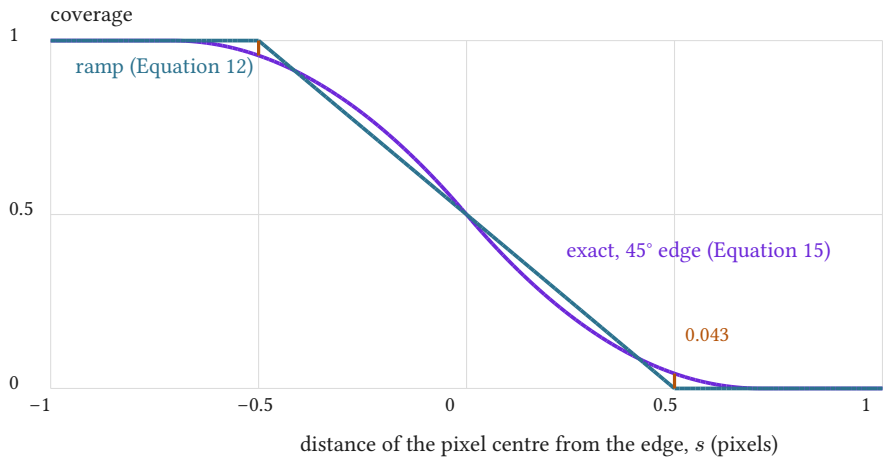



Figure 3: Pixel coverage against the signed distance $s$ of the pixel centre from the edge, in pixels. The linear ramp the mesh draws (blue) and the exact coverage of a 45° edge (violet); the largest gap, 0.043, is at $|s| = \frac{1}{2}$.

| Edge angle | 0° | 15° | 30° | 45° |
|---|---|---|---|---|
| Largest error of the ramp | 0 | 0.025 (6.4/255) | 0.039 (9.9/255) | 0.043 (10.9/255) |

The table is computed by clipping a unit square against the half-plane and comparing with Equation 12 on 2,001 values of $s$. In practice most edges in a glyph are curved and spread over all angles, so the typical error is far below this worst case (Table 3).

#### *8.5.4 Why the inset must be exactly half*

The inset fraction $f$ is the part of the one-pixel ring that lies inside the true edge ($f = \frac{1}{2}$ above). `INSET_FRACTION` is a constant in `src/vector.rs`, and it was swept:

| Inset $f$ | 0.2 | 0.35 | 0.5 | 0.6 | 0.7 | hard edge |
|---|---|---|---|---|---|---|
| Mean error, 12 cases | 9.81 | 5.29 | **2.16** | 3.77 | 5.97 | 7.50 |
| at 1× | 16.19 | 8.97 | **4.53** | 7.22 | 10.28 | 12.65 |
| at 2× | 7.89 | 4.09 | **1.21** | 2.47 | 4.48 | 5.97 |
| at 3× | 5.34 | 2.81 | **0.75** | 1.63 | 3.14 | 3.88 |

Table 2: Mean absolute alpha difference (0 to 255) against Typst's own raster, for four formulas at three scales, against the inset fraction. The symmetric ramp wins on every scale, and an inset of 0.2 is worse than no feathering at all (9.81 against 7.50).

**THE MATH**

The result matches the theory. An inset smaller than $\frac{1}{2}$ leaves the opaque interior too large (heavy edges); a larger one eats into the shape (thin edges). At $f = \frac{1}{2}$ the ramp is centred on the true edge, so a sample on the edge reads 1/2, which is what area coverage says for a straight edge.

#### *8.5.5 Where it is weakest: strokes thinner than a pixel*

Insetting a contour by $\frac{1}{2}$ px on both sides leaves a stem of width $w$ with width $w - 1$ px. When $w < 1$ px the inset polygon turns inside out, and the non-zero rule fills the inverted sliver as if it were interior, at full opacity. A hairline serif at a small size is such a stem, so small text at 1 pixel per point is the case where the vector backend departs most from Typst's rasteriser (the "1×" row above). The result is a stem that is too bold, not too faint. At 2 px/pt and above (every high-density display) the same strokes are 2 px or wider in the places that matter and the effect all but disappears.

**MEASURED**

Summing the alpha of the CPU-rasterised mesh and of Typst's raster for the same snippet: at 1× the feathered mesh draws **+19.0%, +20.7% and +18.9%** more ink on the TVaR, the prose and the square-root formulas; at 2× it is +2.9%, +2.7% and +0.1%; at 3×, +1.2%, +1.5% and +1.5%.

### 8.6 How these claims were measured

#### *8.6.1 A rasteriser for meshes*

To compare a mesh with Typst's raster without a GPU, `egui_typst::debug::rasterise_alpha` draws it on the CPU following the rule of a GPU without multisampling. For a triangle with corners $p_0, p_1, p_2$ and a pixel centre $c$, the barycentric weights are

$$w_0 = \frac{(p_1 - c) \times (p_2 - c)}{(p_1 - p_0) \times (p_2 - p_0)}, \quad w_1 = \frac{(p_2 - c) \times (p_0 - c)}{(p_1 - p_0) \times (p_2 - p_0)}, \quad w_2 = 1 - w_0 - w_1 \tag{16}$$

with $a \times b = a_x b_y - a_y b_x$. The pixel is drawn if $w_0, w_1, w_2 \geq 0$; its opacity is the weighted mean of the three vertices', blended onto the canvas with Equation 5.

#### *8.6.2 The metric*

The difference between two pictures $a$ and $b$ of $W \times H$ pixels is the mean absolute difference of their alpha channels,

$$D(a, b) = \frac{1}{WH} \sum_{x,y} |a_{xy} - b_{xy}|, \quad 0 \leq D \leq 255 \tag{17}$$

so $D = 1$ means that pixels differ by about 0.4% of full scale on average. It is deliberately simple: no perceptual weighting, and any mismatch of position shows up.

| | hard edges | | | feathered | | |
|---|---|---|---|---|---|---|
| Formula | 1× | 2× | 3× | 1× | 2× | 3× |
| $\mathrm{TVaR}_\alpha(L) = \ldots \mathrm{d}u$ | 7.6 | 3.6 | 2.7 | 2.92 | 0.71 | 0.45 |
| $\Sigma = \left(\begin{smallmatrix} 1 & \rho_{12} \\ \rho_{12} & 1 \end{smallmatrix}\right)$ | 8.4 | 4.6 | 2.9 | 2.77 | 0.73 | 0.55 |
| Loss $L = -$result has CoV $= \frac{\sigma}{\mu}$ | 19.0 | 8.2 | 6.1 | 7.58 | 1.84 | 1.20 |
| $\sqrt{x^2 + y^2} \geq 0$ | 15.6 | 7.4 | 3.9 | 4.86 | 1.57 | 0.78 |

Table 3: $D$ of Equation 17 between the vector mesh (CPU-rasterised) and Typst's raster. Hard edges improve with scale, as the stair-steps shrink with the pixel (2.7 to 19); the feathered edge is lower at every scale, 2.5 to 6.2 times lower, from about 3–8 at 1× to about 0.5–1.8 at 2× and 3×. The test asserts less than 2.5 at 2× and above, less than 8 at 1×, and that feathering lowers the error in every case.

**MEASURED**

The `Svg` backend, rasterised with resvg and compared the same way at 2×, differs from Typst's raster by $D = 0.70$, 0.85 and 2.09 on three of these formulas, so `Vector` at 2× (0.71, 0.73, 1.84) is as close to the reference as the SVG route is. Through egui's own loader (a frame run, no GPU), the SVG backend uploads a texture of the expected size whose visible pixels are the requested colour in at least 95% of cases, and both backends draw the same amount of ink to within 1% (5,118 against 5,074, the sum of alpha over the texture).

**PITFALL**

**A comparison that does not move when it should is a warning.** The first "hard edge" figures in this project were 11.9, 11.9 and 12.0 at 1×, 2× and 3×: constant, because the mesh was empty (a zero tolerance made the triangulator refuse the fill) and the number was the error of a blank picture. Aliasing error must shrink as pixels do. The corrected figures (Table 3) do, and `tests/vector.rs` now asserts both that a hard-edged mesh draws roughly the reference's amount of ink and that its error falls with scale.

**TIP**

To check a change to the vector code, run `cargo test --release --test vector -- --nocapture`. It prints the table of Table 3. A change that moves a number by more than a tenth of a unit at 2× deserves a look with `render_png --wire`.

# 9 Reference

## 9.1 The public interface

| Item | What it is |
|---|---|
| `Typst::new(&str)` | The widget. `.size(pt)`, `.color(Color32)`, `.backend(Backend)`; implements `egui::Widget`. |
| `Backend` | `Svg` (default), `Vector`, `Raster`. |
| `Compiler` | Fonts, library and one main file. `Compiler::new()`; `svg(snippet, size_pt, margin_pt)`, `vector(...)` and `render(snippet, &Style)`. `Send + Sync`; compiles are serialised. |
| `Style` | `color`, `size_pt`, `margin_pt`, `pixels_per_point` for `render`. |
| `Svg` | `text`, `size_pt`, and `tinted(Color32)` which returns the SVG with its text in that colour. |
| `Vector` | `size_pt`, `skipped`, `contour_count()`, `mesh(pixels_per_point, colour)` and `mesh_with_feather(feather_pt, colour)` (`0.0` gives hard edges). |
| `render(snippet, &Style)` | A one-call form using a process-wide `Compiler` (built on first use). |
| `escape(&str)` | Backslash-escapes ASCII punctuation so that text is shown literally. For markup, not for math. |
| `Error` | As in Section 6. |
| `MAX_SIDE_PX`, `MAX_PIXELS`, `SENTINEL` | 8192; $2^{25}$; `#010203`, the placeholder text colour of `Svg` and `Vector`. |

## 9.2 What the vector backend does not draw

Images, gradients and tilings, colour glyphs (emoji), clipped groups, dashed strokes and even-odd fills are not drawn; each is counted in `Vector::skipped`. Math needs none of them: for every snippet in the test suite `skipped` is 0. Strokes are drawn as one quad per straight segment with butt caps, which is exact for the rules of fractions and radicals.

## 9.3 Known limitations and what is next

- **No text selection and no accessible name.** A formula is an image (or a mesh) to egui; a screen reader finds nothing. The plain-text form of each formula is the obvious label.
- **No inline flow.** A snippet is one block. `Typst::new("a loss $L$ has mean $mu$")` is a single image: it does not wrap with egui's text.
- **The margin is fixed** at 2 pt in the widget (`Compiler` takes it as a parameter).
- **The cache clears whole** at 256 entries (Section 5.3), and compiles happen on the calling thread.
- **Only the bundled fonts**; other scripts draw boxes.
- **1× small text** is the weakest case of the vector backend (Section 8.5).

## 9.4 Rebuilding this manual

```
sh manual/figures.sh                  # figures from render_png
typst compile manual/egui_typst.typ   # writes manual/egui_typst.pdf
```

The source uses only the fonts that ship with the compiler. The formulas in the text are typeset by Typst, the same compiler that `egui_typst` embeds; the pictures of output (Figure 1, Figure 2) come from the crate itself, and the chart in Figure 3 is drawn here from Equation 15 and Equation 12.